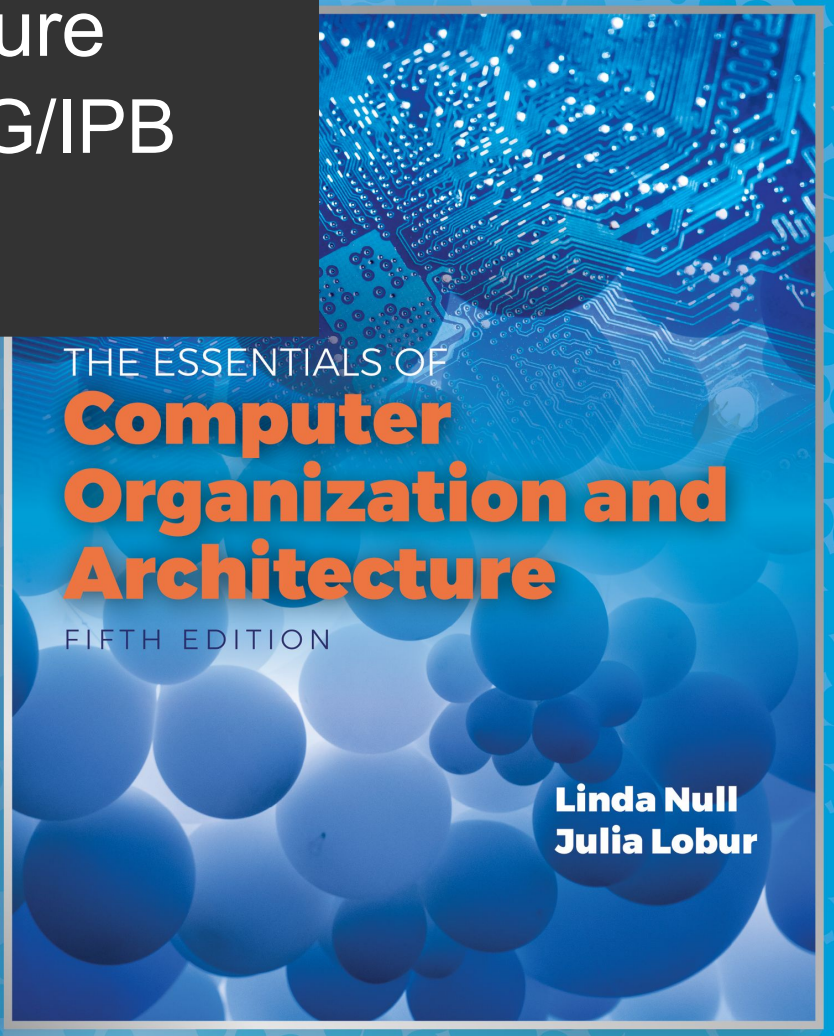
Computer Architecture
Informatics Eng., ESTiG/IPB
2024/2025

José Rufino

(mainly based on Chapter 5 of the book "The Essentials of Computer Organization and Architecture" of Linda Null & Julia Lobur)



Unit 4:
Instruction Set Architectures

Objectives

- Understand the factors involved in the design of *Instruction Set Architectures (ISAs)*
- Know the diversity of *Memory Addressing Modes*
- Understand the concept of *Instruction-level Pipelining* and its affect on the performance of programs
- Learn the properties that distinguish RISC from CISC architectures

4.1 Introduction

- This chapter builds upon the concepts previously introduced by the study of the MARIE architecture
- It presents a detailed look at different *instruction formats*, *operand types*, and *memory access methods*
- It covers the interrelation between *machine organization* and *instruction formats*
- Result: a deeper understanding of computer architectures in general

4.2 Instruction Formats
- Design Decisions

Instruction sets are differentiated by:

- Number of bits per instruction (MARIE: 16 = 4 + 12)
- Type of storage of instructions operands inside the CPU: Stack-based or register-based (MARIE)
- Number of explicit operands per instruction (MARIE: 1)

4.2 Instruction Formats
- Design Decisions

Instruction sets are differentiated by (cont.):

- Operand location (reg-to-reg, reg-to-mem, mem-to-reg, mem-to-mem; MARIE ? (input, output))
- Types of operations (including the ones that access RAM memory; MARIE: inst X)
- Type and size of operands (these may be addresses, numbers, characters; MARIE ? (IO))

4.2 Instruction Formats
- Design Decisions

Inst. Set Architectures also comparable according to:

- Main memory space occupied by a program
- Instruction complexity
  - Amount of work in the decoding phase
  - Complexity of the tasks to be executed
- Instruction length (in bits)
- Total number of instructions in the instruction set

4.2 Instruction Formats
- Design Decisions

In **designing an instruction set**, consider:

- Instruction length (short/long, fixed/variable)
  - Short: less RAM, faster fetch, less instructions, smaller operands, less operands per instruction
  - Fixed: easier decoding, wasted bit space
- Instruction length vs Number of operands
  - Fixed size does not imply fixed number of operands ...
- Which addressing modes to support
  - Direct (MARIE), indirect (MARIE), indexed, ...

4.2 Instruction Formats
- Design Decisions

In **designing an instruction set**, consider:

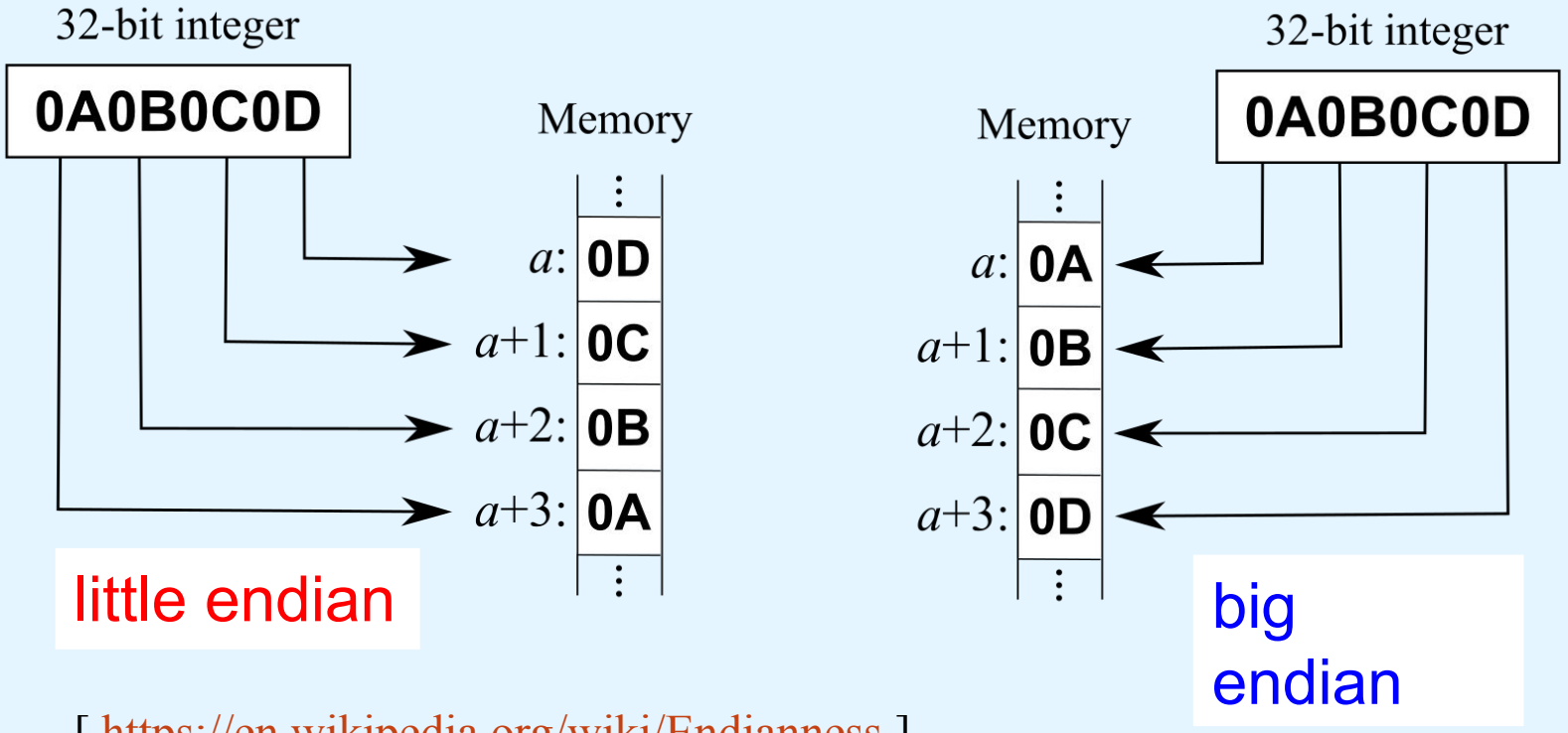
- Memory organization
  - whether byte- or word- addressable
  - memories with words > 1 byte (16/32/64 bits) often support byte-addressing instead of word-addressing; this allows to have instructions that handle individual characters in a string
- Number, organization and role of registers
  - generic vs specific, register windows, …
- Addressing/Storage order of the bytes of a word
  - endianness: big endian vs little endian (see next)

4.2 Instruction Formats
- Endianness

- In most modern architectures memory is byte-addressed; thus, byte ordering, or ***endianness***, is a major decision
  - how is a many-byte integer stored in main memory ? or how the same integer is sent through a network ?
- **Little endian** approach: the least significant byte is stored/sent first, followed by the most significant bytes
  - example: Intel, ARM CPUs; SMB network protocol
- **Big endian** approach: the most significant byte is stored/sent first, followed by the least significant bytes
  - example: Motorola, MIPS, PowerPC CPUs; TCP/IP networks

4.2 Instruction Formats
- Endianness

- Example: memory storage of a 32 bit (4-byte) integer



[https://en.wikipedia.org/wiki/Endianness]

4.2 Instruction Formats
- Endianness

checkpoint: exercises LEBE1 to LEBE7

- **Big endian**:
  - More natural (akin to digit insertion order in an ATM machine)
  - Sign determined by looking at the byte at address offset 0
  - Strings and integers are stored in the same order

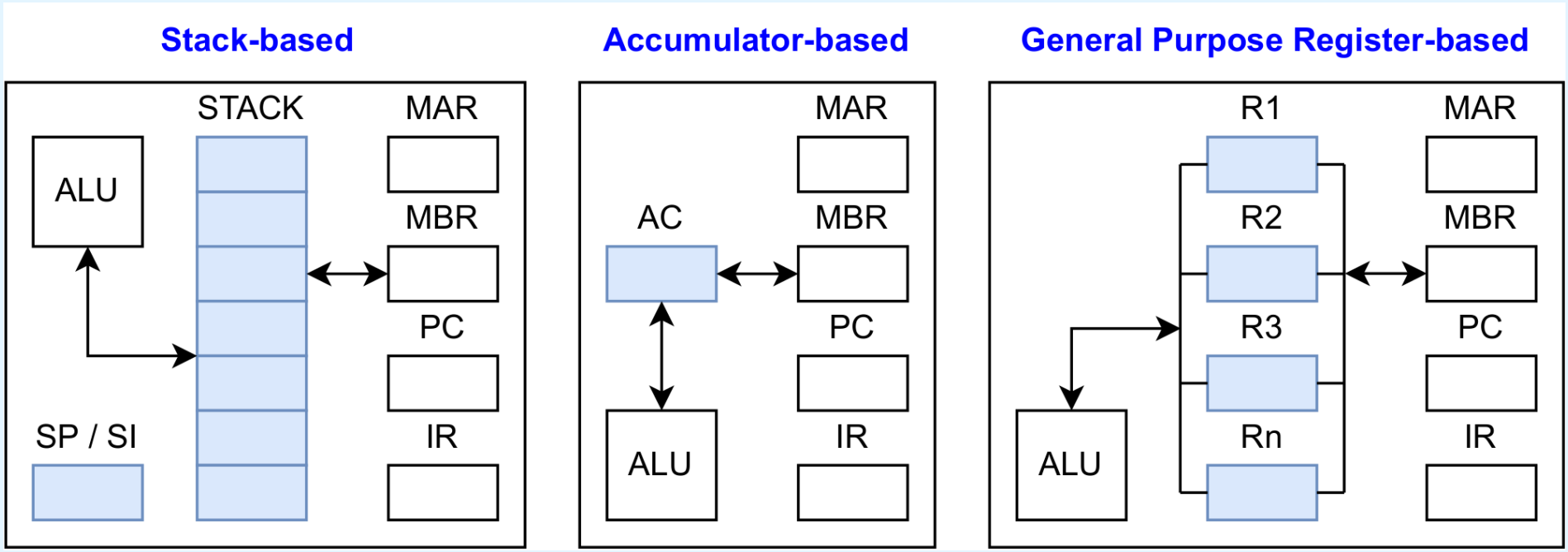
- **Little endian:**
  - Makes it easier to place values on non-word boundaries.
  - Conversion from a 16-bit integer address to a 32-bit integer address does not require any arithmetic (only truncation):
    - number 0xMMNNmmnn; BE stores MM NN mm nn; LE stores nn mm NN MM; with LE, is enough to keep the 1st 2 bytes in order to convert from 32 bits to 16 bits
  - Readings of a wrong size (e.g., in a type cast) may still produce correct values
    - number 0x0000**5678**; BE stores 00 00 56 78; LE stores **78 56** 00 00; with LE, reading by mistake just the 1st 2 bytes still gets the right values

4.2 Instruction Formats
- Internal CPU Storage

- The next consideration for architecture design concerns **how the CPU stores data internally**
- There are three main approaches:
  1. A **stack** -based architecture
  2. An **accumulator** -based architecture
  3. A **general purpose register** -based architecture
- In choosing one approach over the other, the trade-offs are simplicity (and cost) of hardware design vs execution speed vs ease of use

4.2 Instruction Formats
- Internal CPU Storage

- Main approaches to **internal CPU storage**:



4.2 Instruction Formats
- Internal CPU Storage

- **Stack architecture** (example: MSP)
  - instructions and implicit operands are taken from the CPU stack (explicit operands still refer to RAM addresses)
  - supports simple models to evaluate expressions (in postfix)
  - a stack cannot be accessed randomly => less efficient code
  - a register SI (Stack Index) identifies the current stack top (this may be the next free cell of the stack, or the last busy cell)

- **Accumulator architecture** (example: MARIE)
  - accumulator register is an implicit operand
  - in a binary operation, the other operand is a memory address
  - a single accumulator register => intense memory bus traffic

4.2 Instruction Formats
- Internal CPU Storage

- **General purpose register** (GPR) **architecture**
  - has one (or more) sets of generic registers
  - keep as much operands as possible in registers => less memory accesses => faster than Accumulator architecture
  - compilers are able to generate more efficient code
  - results in longer instructions, once they need fields to identify registers => slowest fetch and decoding
  - nowadays, most systems follow a GPR architecture

4.2 Instruction Formats
- Internal CPU Storage

- **Types of GPR-based systems**, by operand location:
  - **memory-memory**: two or three operands in memory
    - DEC VAX **Add X,Y** // X = X + Y
  - **register-memory**: one operand in register, other in memory
    - Intel, Motorola (CISC …), MARIE
    - Load R,X; **Add R,Y**; Store R,X // X = X + Y
  - **load-store**: no operands in memory (all in register)
    - SPARC, MIPS, ALPHA, PowerPC (RISC …)
    - Load R1,X; Load R2,Y; **Add R1,R2**; Store R1,X // X=X+Y
- The number of operands and the number of available registers has a direct affect on instruction length →

4.2 Instruction Formats
- Number of Operands and Instruction Size

- Fixed size instructions (example: MARIE):
  - fast decoding (simplicity), fast execution (*pipelining*), space waste (some instructions will have bits unused)
- Variable size instructions:
  - slow decoding (complexity), more efficient space usage
- Usual scenarios:
  - co-existence, in the same architecture, of 2 or 3 instruction sizes, using bit patterns crafted to ease the decoding process
  - instruction size matches word size => perfect memory alignment => faster access, less waste of memory
    - variable size instructions => memory waste is inevitable, once instructions are always word-aligned

example: 24 bits instructions stored in 32 bit words

4.2 Instruction Formats
- Number of Operands and Instruction Size

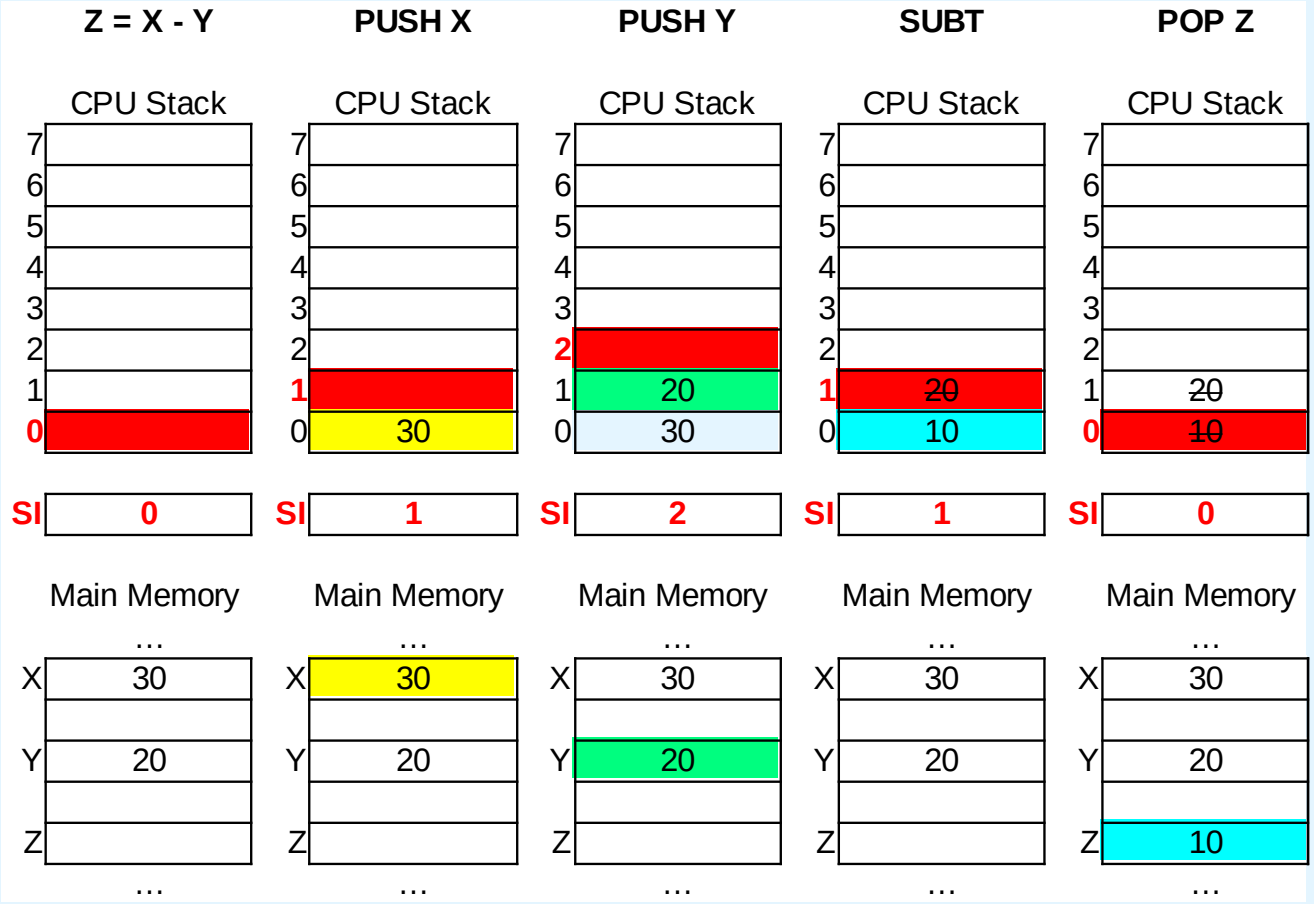
- **Common instruction formats**:
  - opcode (zero addresses)
  - opcode + 1 address (typically a RAM address; **MARIE**)
  - opcode + 2 addresses
    - 2 registers
    - 1 register + 1 RAM address
  - opcode + 3 addresses
    - 3 registers
    - combinations of register and RAM addresses

4.2 Instruction Formats
- Number of Operands and Instruction Size

- **Stack-based architectures:**
  - instructions with one or zero explicit operands
  - memory acess: "**`PUSH X`**" and "**`POP X`**"
    - one explicit parameter (**`X`**) is a main memory address, one implicit parameter is the top of the stack (*STACK[SI]*)
    - **`PUSH X`** : copies M[X] to the top of the stack and grows the stack (*STACK[SI]=M[X]; SI* ++); **`POP X`** : copies the top of the stack to M[X] and shrinks the stack (*M[X] = STACK[SI-1]; SI*--)
  - binary arithmetic instructions: "**`ADD`**", "**`ADD`**", ...
    - operands and results implicit on the top of the stack
    - **`SUBT`** : removes the two topmost stack values, and inserts their difference (*STACK[SI-2]=STACK[SI-2] - STACK[SI-1]; SI*--)

4.2 Instruction Formats
- Number of Operands and Instruction Size

- **Stack-based architectures (cont.):**
  - example of evaluation of an expression (Z = X - Y)



4.2 Instruction Formats
- Number of Operands and Instruction Size

- **Stack-based architectures (cont.):**
  - these architectures require to think a little differently
  - we are accustomed to writing expressions using the ***infix*** notation, such as: **Z = X + Y**
  - stack arithmetic requires a ***postfix*** notation: **Z = XY+**
    - parentheses are not used, and translation to assembly of a stack-based architecture is a trivial task (see next slide)
  - some infix expressions and their postfix equivalents:

(X + Y) × (W − U) + 2	Z = (X × Y) + (W × U)
X Y + W U − × 2 +	Z = X Y × W U × +

checkpoint: exercises SBA1 to SBA3

4.2 Instruction Formats
- Number of Operands and Instruction Size

- examples with different number of explicit operands to evaluate the same expression (considering only arithmetic operations):
  - in a stack-based (0-address) ISA, the *postfix* expression

    Z = X Y × W U × +

    can be evaluated by the code

```
PUSH X
PUSH Y
MULT
PUSH W
PUSH U
MULT
ADD
POP Z
```

4.2 Instruction Formats
- Number of Operands and Instruction Size

- examples with different number of explicit operands to evaluate the same expression (cont.):
  - in a 1-address ISA, like MARIE, the *infix* expression

    Z = X × Y + W × U

    can be evaluated by the code

```
LOAD X
MULT Y
STORE TEMP
LOAD W
MULT U
ADD TEMP
STORE Z
```

4.2 Instruction Formats
- Number of Operands and Instruction Size

- example of using different number of explicit operands to evaluate the same expression (cont.):
  - in a 2-address ISA (Intel, Motorola), the *infix* expression

    Z = X × Y + W × U

    can be evaluated by the code:

```
LOAD  R1,X
MULT  R1,Y
LOAD  R2,W
MULT  R2,U
ADD   R1,R2
STORE Z,R1
```

Note: 2-address ISAs usually require one operand to be a register, as in this example (R1,R2)

4.2 Instruction Formats
- Number of Operands and Instruction Size

- example of using different number of explicit operands to evaluate the same expression (cont.):
  - in a 3-address ISA (mainframes), the *infix* expression

    Z = X × Y + W × U

    can be evaluated by the code:

```
MULT R1,X,Y
MULT R2,W,U
ADD  Z,R1,R2
```

checkpoint: exercises NOIS1 to NOIS5

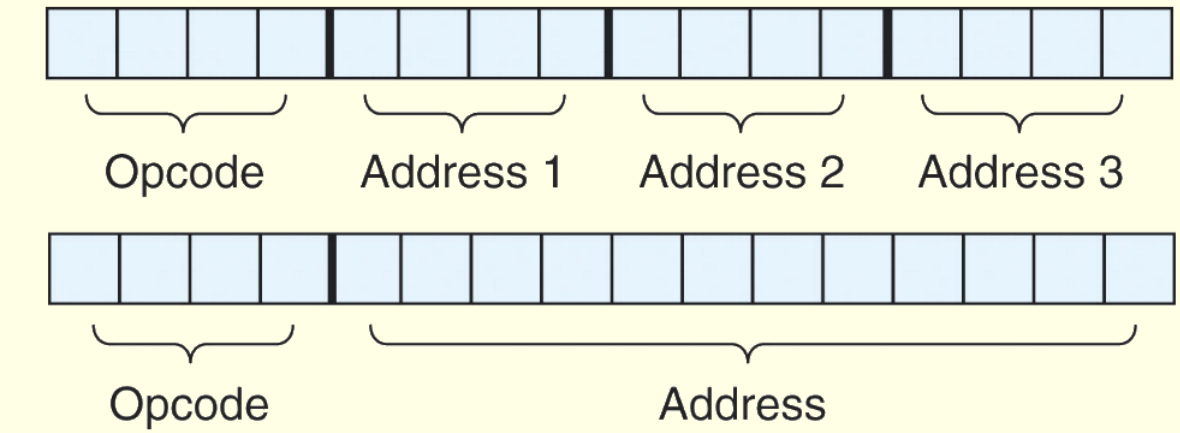
Having less instructions, would this program execute faster than the corresponding (longer) program that we saw for the stack-based ISA?

4.2 Instruction Formats
- Expanding Opcodes

- We have seen how instruction length is affected by the number of operands supported by the ISA
- In any instruction set, not all instructions require the same number of operands
- Operations that require no operands, such as **`HALT`**, necessarily waste some space when fixed-length instructions are used
- One way to recover some of this space is to use ***expanding opcodes***

4.2 Instruction Formats
- Expanding Opcodes

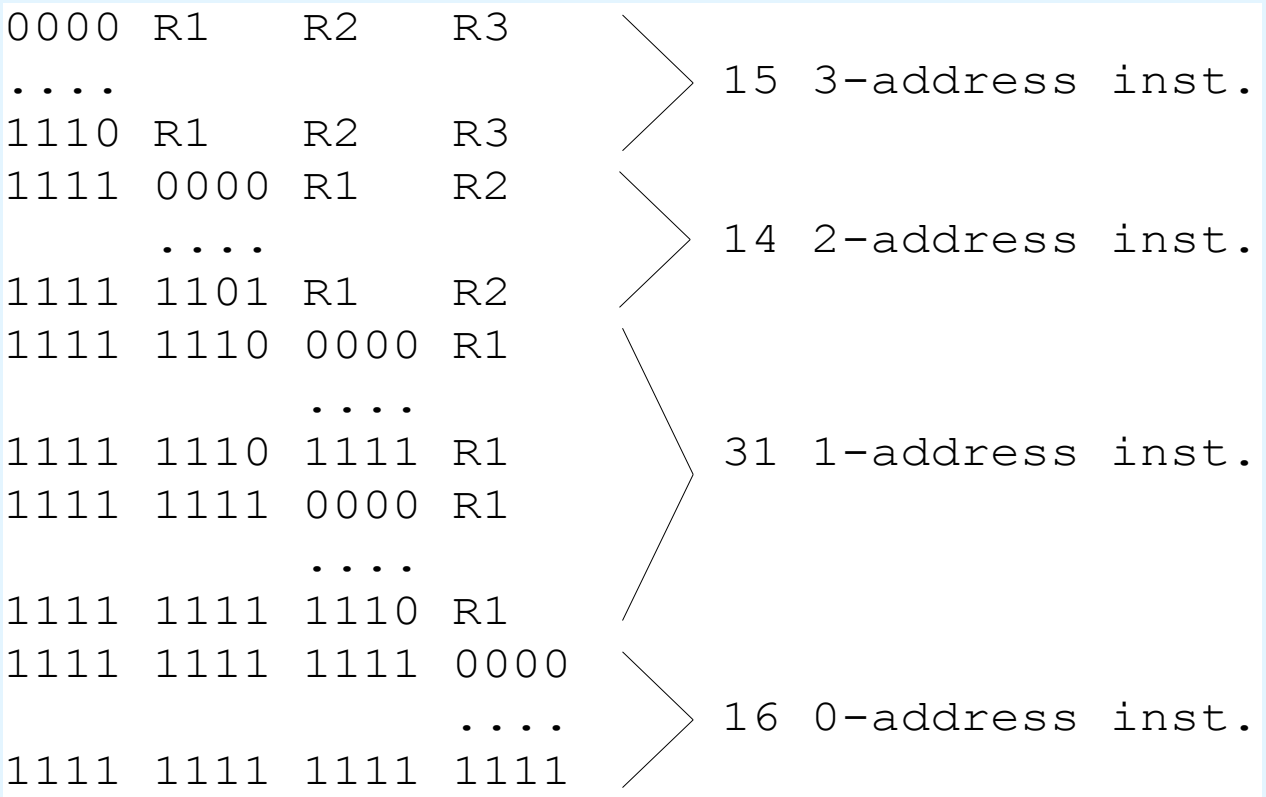
- Consider a system with 16 registers and 4K of RAM.
  - we need 4 bits to identify each different register
  - we need 12 bits for each different memory address
- If the system is to have 16-bit instructions, and the **opcode has fixed length**, some possible instruction formats are:



  - up to 16 instructions with 3 registers ids. as operands
  - up to 16 instructions with 1 memory address operand

4.2 Instruction Formats
- Expanding Opcodes

- If the **length of the opcode may vary**, we can create a very rich instruction set, with more than 16 instructions:



4.2 Instruction Formats
- Expanding Opcodes

checkpoint:
exercises EO1 to EO4

- Decoding would now take some extra work; but that could be minimized by a carefully crafted algorithm:

```
if (leftmost four bits != 1111 ) {
  Execute appropriate three-address instruction
}
else if (leftmost seven bits != 1111 111 ) {
  Execute appropriate two-address instruction
}
else if (leftmost twelve bits != 1111 1111 1111 ) {
  Execute appropriate one-address instruction
}
else {
  Execute appropriate zero-address instruction
}
```

4.3 Instruction types

Instructions fall into several broad categories:

- Data movement (between registers and memory)
- Arithmetic (integer and floating point)
- Boolean (AND, OR, XOR, NOT)
- Bit manipulation (arithmetic, logic, shifting, rotate)
- I/O (programmed IO, interrupt-driven IO, DMA IO)
- Control transfer (jumps, skips, function call and return)
- Special purpose (string processing, cache management, ..)

Can you think of some examples of each of these in MARIE?

4.4 Addressing
- Addressing Modes

- **Addressing modes**
  - Ways of defining where an operand is located
  - May specify
    - a constant,
    - a register,
    - or a memory location
  - *Effective address:* the actual location of an operand
  - Certain addressing modes allow to determine the address of an operand *dynamically*

4.4 Addressing
- Addressing Modes

- ***Immediate*** *addressing*
  - the data (operands) is embedded in the instruction
  - optimal performance, none flexibility
  - NOT 008 ==> 008 is the value to be negated
- ***Direct*** *addressing* (MARIE)
  - the effective (memory) address of the data is given in the instruction
  - fair performance, some flexibility
  - NOT 008 ==> 008 is the (memory) address of the value to negate
- ***Register Direct*** *addressing*
  - similar to Direct addressing, but using CPU registers
  - data is located in a register, whose id is embedded in the instruction
  - NOT 008 ==> 008 is the id of the CPU register with the value to negate

4.4 Addressing
- Addressing Modes

- ***Indirect*** *addressing* (MARIE)
  - the instruction includes the address of a memory position
  - on that position it is stored the effective address of the data
  - the address in the instruction is the location of a pointer
  - NOT 008 ==> negate the value whose address is in 008

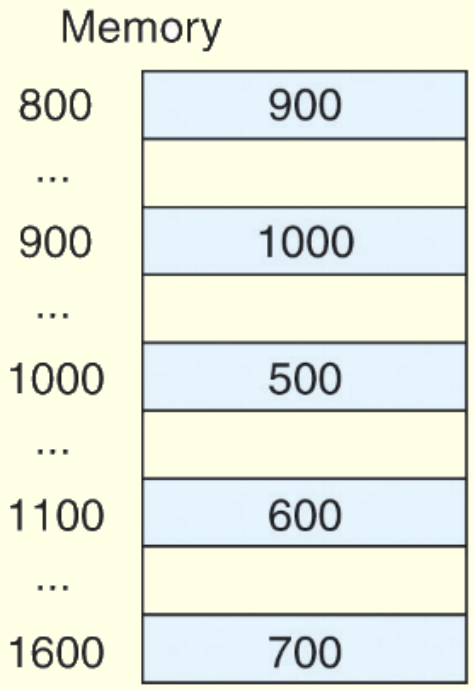
- ***Register Indirect*** *addressing*
  - similar to Indirect addressing, but using CPU registers
  - a register stores the effective (memory) address of the data
  - NOT 008 ==> negate the value whose address is in register 008

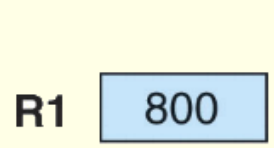
4.4 Addressing
- Addressing Modes

- ***Indexed*** *addressing*
  - the instruction includes a memory address
  - a register (implicit or explicit) holds an offset
  - data effective address = memory address + offset
  - NOT 008 ==> negate the value whose address is 008+R1
- ***Based*** *addressing*
  - opposite to Indexed addressing: a base register stores a memory address, and the instruction holds the offset
- ***Stack*** *addressing*
  - operand(s) assumed to be on the top of the stack
- There are many other variants, not covered here:
  indirect indexed, self relative, auto increment/decrement, ...

4.4 Addressing
- Addressing Modes

- For the instruction shown, what value is loaded into the accumulator for each addressing mode? (R1 is used a index register if necessary)





LOAD 800

Mode	Value Loaded into AC
Immediate	800
Direct	900
Indirect	1000
Indexed	700

checkpoint: exercises AM1 to AM3

4.5 Instruction-Level Pipelining

- Some CPUs divide the fetch-decode-execute cycle into smaller steps or stages
- These stages can often be executed in parallel to increase **throughput** (number of instructions finished per time unit)
- Such parallel execution is called instruction-level pipelining
- This term is sometimes abbreviated ILP in the literature
- ILP is similar to an industrial assembly line divided in stages
  - several stages happen at the same time
  - each stage feed from the previous, and feeds the next stage

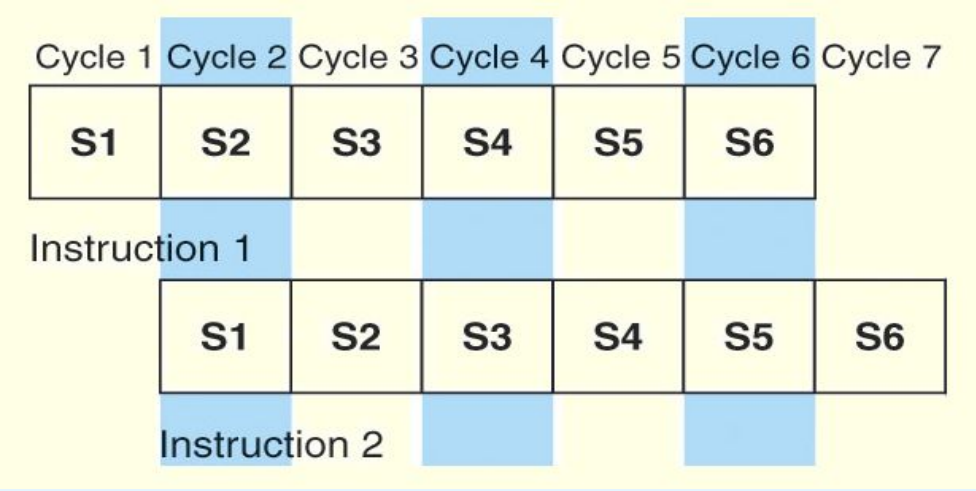
4.5 Instruction-Level Pipelining

- **Example**: suppose a fetch-decode-execute cycle broken into the following smaller steps:

  1. **Fetch instruction**
  2. **Decode opcode**
  3. **Calculate effective address of operands**
  4. **Fetch operands**
  5. **Execute instruction**
  6. **Store result**

  – there's a pipeline with k=6 stages (Sk), one per step

  – for every clock cycle, one step is carried out, and the k=6 stages are overlapped ==>



4.5 Instruction-Level Pipelining

- Speedup offered by a pipeline:
  - consider a pipeline with k stages of equal duration t_s
  - each stage takes t_s units of time (pipeline clock cycle)
  - consider n instructions that are going trough the pipeline
  - the first instruction requires $t_i = k \,.\, t_s$ time to complete
  - the other $n - 1$ instructions leave the pipeline one per cycle
  - the time to complete the remaining instructions is $(n - 1).t_s$
  - to complete n instructions using a k-stage pipeline requires:

$(k \,.\, t_s) + (n - 1).t_s = (k + n - 1).t_s$ [in units of time or clock cycles]

4.5 Instruction-Level Pipelining

- Speedup offered by a pipeline (cont.):

  - to complete ***n*** instructions without pipelining would require:

$$n \cdot t_i = n \cdot (k \cdot t_s)$$

  - speedup (acceleration) provided by pipelining ***n*** instructions: the time without a pipeline, divided by the time with pipeline

$$S_n = n \cdot t_i / (k + n - 1) \cdot t_s = n \cdot (k \cdot t_s) / (k + n - 1) \cdot t_s$$
$$= n \cdot k / (k + n - 1)$$

  - if we take the limit as ***n*** approaches infinity, ***(k + n - 1)*** approaches ***n***, which results in a **maximum speedup** of ***k***:

$$MAX(S_n) = S_{n \to \infty} = n \cdot k / (k + n - 1) = k$$

checkpoint: exercises ILP1 to ILP3

4.5 Instruction-Level Pipelining

- The former speedup formulas make several assumptions:
  - Each pipeline stage takes the same time
  - The architecture supports fetching instructions and data in parallel (which implies separate buses)
  - The pipeline can be kept filled at all times, which is not always the case; this happens, for instance, when
    - it is uncertain the outcome of a condition test (which instructions should enter, in advance, into the pipeline ? those of the “true” path or those of the “false” path ?)
    - instructions in the pipeline need the same resources
    - an instruction still in the pipeline depends on the result of another that is still also in the pipeline (not yet finished)

4.5 Instruction-Level Pipelining

- A pipeline may stall, or be flushed, for several reasons:
  - conditional branching

Time Period →	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction: 1	S1	S2	S3	S4									
2		S1	S2	S3	S4								
(branch) 3			S1	S2	S3	S4							
4				S1	S2	S3							
5					S1	S2							
6						S1							
8							S1	S2	S3	S4			
9								S1	S2	S3	S4		
10									S1	S2	S3	S4	

  - solutions: **i)** branch prediction (CPU), **ii)** *fetch* of instructions for both alternatives (CPU), **iii)** jump delaying (by reordering or insertion of other instructions; this is done by compilers)

4.5 Instruction-Level Pipelining

- A pipeline may stall, or be flushed, for several reasons:
  - resource conflicts
    - example: simultaneous memory bus use by LOAD and STORE
    - solutions: **i)** the instruction behind in the pipeline will wait for the resource to become free (or other instructions are reordered / inserted to keep the pipeline "busy"); **ii)** increase the resource units
  - data dependencies
    - a instruction depends on a result not yet produced by another
    - solutions: **i)** hardware that detects the situation and generates NOOP instructions to feed the pipeline; **ii)** instruction reodering/insertion
- However, these hazards cannot be totally eliminated ...

4.6 ILP vs Other Acceleration Techniques

- Instruction Level Parallelism (ILP):
  - simultaneous execution of several instructions within the same CPU core; implemented (**automatically**) at the hardware (CPU), using techniques like i) (*super-*)*pipelining* and ii) *super-scalarity*
- *Super-Pipelining:* the CPU has more than one pipeline
- *Super-Scalarity*: the CPU has multiple functional units of the same kind (e.g., ALUs); often combined with super-pipelining
- *Explicit Paralell Instruction Computers* (EPIC): several small instructions that can be executed in parallel packed together in a macro-instruction during compilation (Intel Itanium 128 bit instr.)
- Program Level Parallelism (PLP): the programmer partitions (**explicitly**) the program in sections to be executed in parallel; requires a parallel system (a multi-core CPU or a computer cluster)

4.7 RISC vs CISC Architectures

- **RISC (Reduced Instruction Set Computer)**
  - i) few different instructions; explicit memory access restricted to some specific instructions (*load* & *store*)
  - ii) all instructions have the same length (nº bits)
  - iii) same nº of clock cycles to decode and execute
- **CISC (Complex Instruction Set Computer)**
  - i) many different kinds of instructions, some with explicit and others with implicit memory access
  - ii) variable instruction length
  - iii) variable *fetch-decode-execute* time

4.7 RISC vs CISC Architectures

- The difference between CISC and RISC becomes evident in the basic computer performance equation:

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- RISC systems reduce execution time by reducing the number of clock cycles per instruction
- CISC systems improve performance by reducing the overall number of instructions of the programs
- Both benefit from higher CPU frequencies (that is, shorter clock-cycles)

4.7 RISC vs CISC Architectures

(*) see section 3.10

- The simple instruction set of RISC machines enables the control units to be hardwired for maximum speed
  - ISA instructions directly implemented at the hardware level (*)
- The more complex+variable instruction set of CISC machines requires micro-code based control units; these are very flexible, but at the cost of some speed
  - ISA instructions translated to several micro-instructions (*)
- With **fixed-length instructions**, the RISC systems lend itself better to **pipelining** and **speculative execution**
- All in all, RISC promises better performance than CISC

4.7 RISC vs CISC Architectures

- Consider the following code fragments:

CISC

```
mov ax, 10
mov bx, 5
mul bx, ax
```

RISC

```
        mov ax, 0
        mov bx, 10
        mov cx, 5
Begin   add ax, bx
        loop Begin ; loops cx times
```

- The total clock cycles for the CISC version is:

  `(2 movs × 1 cycle) + (1 mul × 30 cycle) = 32 cycles`

- While the clock cycles for the RISC version is:

  `(3 movs × 1 cycle) + (5 adds × 1 cycle) +`
  `(5 loops × 1 cycle) = 13 cycles`

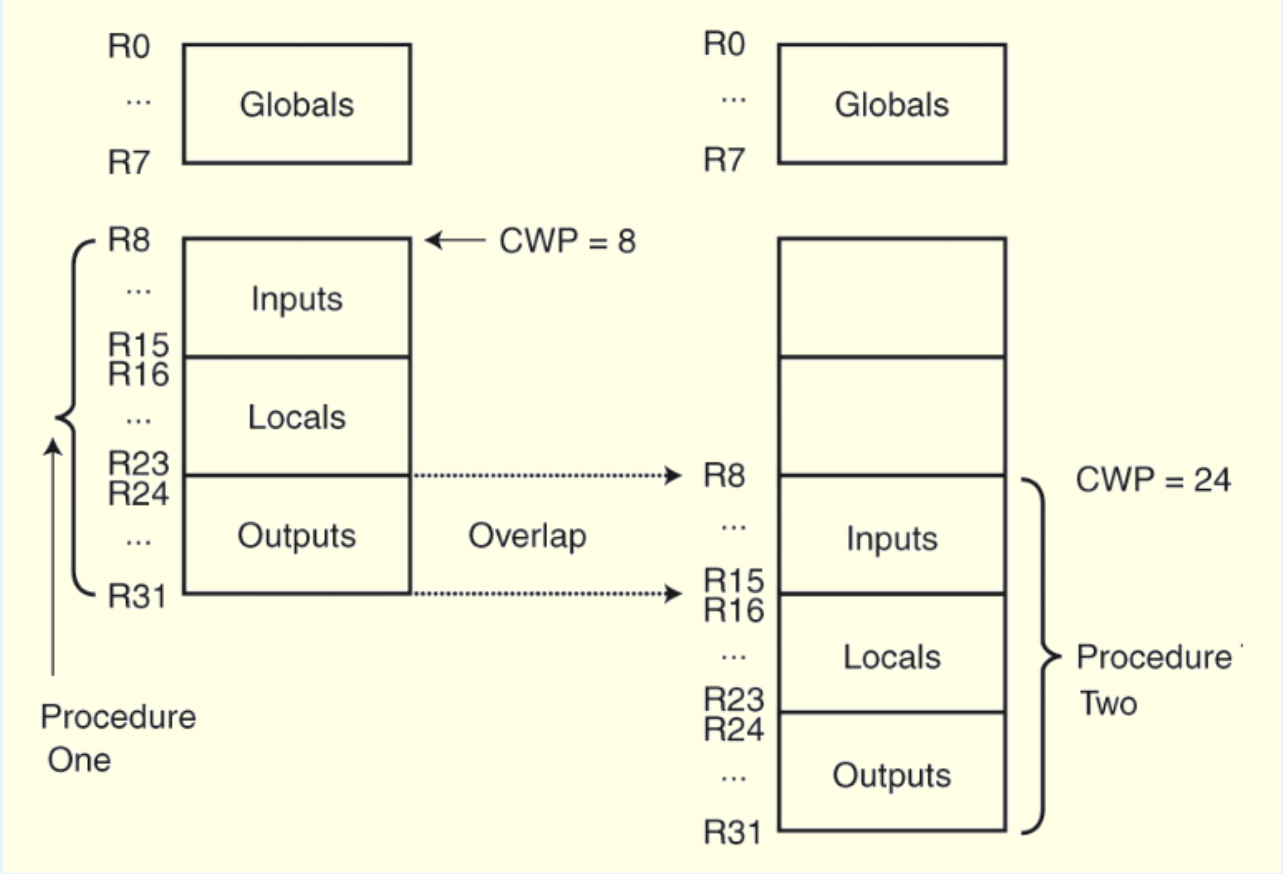
- Needing less cycles, and with shorter cycles (if higher freqs), RISC may offer much faster executions speeds

4.7 RISC vs CISC Architectures

- Because of their *load-store* ISAs, RISC architectures require a large number of CPU registers
- These registers provide fast access to data during sequential program execution
- They can also help to reduce the overhead typically caused by passing parameters to subprograms
  - Instead of passing parameters through a stack in main memory (or by explicit copy between variables, as in MARIE), the caller leaves them in registers, where they are readily accessible to the callee

4.7 RISC vs CISC Architectures

- The registers may overlap when several routines are called (output of a routine is input of other)
- A Current Window Pointer (CWP) points to the active register window



typical values in RISC CPUs:
16 windows, of 32 registers each

4.7 RISC vs CISC Architectures

- RISC vs CISC

RISC	CISC
Multiple register sets, often consisting of more than 256 registers	Single register set, typically 6 to 16 registers total
Three register operands allowed per instruction (e.g., `add R1, R2, R3`)	One or two register operands allowed per instruction (e.g., `add R1, R2`)
Parameter passing through efficient on-chip register windows	Parameter passing through inefficient off-chip memory
Single-cycle instructions (except for `load` and `store`)	Multiple-cycle instructions
Hardwired control	Microprogrammed control
Highly pipelined	Less pipelined
Simple instructions that are few in number	Many complex instructions
Fixed length instructions	Variable length instructions
Complexity in compiler	Complexity in microcode
Only load and store instructions can access memory	Many instructions can access memory
Few addressing modes	Many addressing modes

Today, many CPUs exhibit / combine both RISC and CISC characteristics.

4.8 Real-World Examples of ISAs

- Intel introduced pipelining to their processor line with its Pentium chip
- The first Pentium had two five-stage pipelines. Each subsequent Pentium processor had a longer pipeline, with the Pentium IV having a 24-stage pipeline
- The Itanium (IA-64) has only a 10-stage pipeline

4.8 Real-World Examples of ISAs

- Intel processors support a wide array of addressing modes
- The original 8086 provided 17 ways to address memory, (mostly variants on the methods presented in this chapter)
- Owing to their need for backward compatibility, the Pentium chips also support these 17 addressing modes
- The Itanium, having a RISC core, supports only one: register indirect addressing with optional post increment

4.8 Real-World Examples of ISAs

- MIPS was an acronym for *Microprocessor Without Interlocked Pipeline Stages*

- The architecture is little endian and word-addressable with three-address, fixed-length instructions

- Like Intel, the pipeline size of the MIPS processors has grown: The R2000 and R3000 have 5-stage pipelines; the R4000 and R4400 have 8-stage pipelines

4.8 Real-World Examples of ISAs

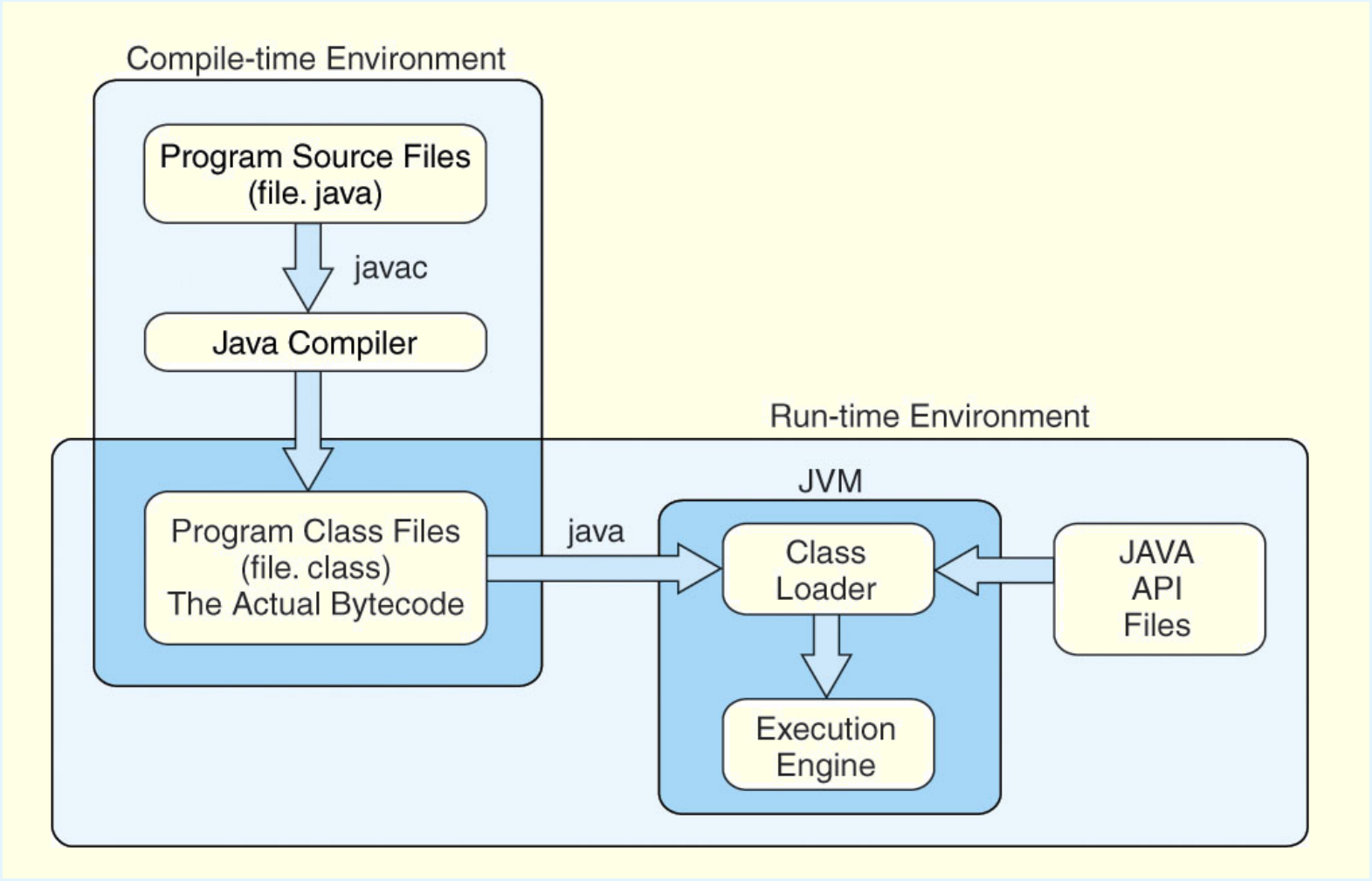
- The R10000 has 3 pipelines: s 5-stage pipeline for integer instructions, a 7-stage pipeline for floating-point instruc., a 6-state pipeline for **`LOAD/STORE`** instructions
- In all MIPS ISAs, only the **`LOAD`** and **`STORE`** instructions can access memory
- The ISA uses only base addressing mode
- The assembler accommodates programmers who need to use immediate, register, direct, indirect register, base, or indexed addressing modes

4.8 Real-World Examples of ISAs

- The Java programming language is an interpreted language that runs in a software machine called the *Java Virtual Machine* (JVM)
- A JVM is written in a native language for a wide array of processors, including MIPS and Intel
- Like a real machine, the JVM has an ISA all of its own, called *bytecode*. This ISA was designed to be compatible with the architecture of any machine on which the JVM is running

The next slide shows how the pieces fit together.

4.8 Real-World Examples of ISAs



4.8 Real-World Examples of ISAs

- Java bytecode is a stack-based language
- Most instructions are zero address instructions
- The JVM has four registers that provide access to five regions of main memory
- All references to memory are offsets from these registers. Java uses no pointers or absolute memory references
- Java was designed for platform interoperability, not performance!

Unit 4 Conclusion

- ISAs are distinguished according to their bits per instruction, number of operands per instruction, operand location and types and sizes of operands
- Endianness is another major architectural consideration
- CPUs can store data internally based on
  1. A stack architecture
  2. An accumulator architecture
  3. A general purpose register architecture

Unit 4 Conclusion

- Instructions can be fixed length or variable length
- To enrich the instruction set for a fixed length instruction set, expanding opcodes can be used
- The addressing mode of an ISA is also another important factor. We looked at:
  - Immediate
  - Register
  - Indirect
  - Based
  - Direct
  - Register Indirect
  - Indexed
  - Stack

Unit 4 Conclusion

- A *k*-stage pipeline can theoretically produce execution speedup of *k* as compared to a non-pipelined machine
- Pipeline hazards such as resource conflicts and conditional branching prevents this speedup from being achieved
- The common distinctions between RISC and CISC systems include RISC's short, fixed-length instructions
- RISC ISAs are load-store architectures. This permit RISC systems to be highly pipelined
- The Intel, MIPS, and JVM architectures provide good examples of the concepts presented in this chapter